SOURCE LISTING

Database Schema Code

AI-Based Examination Platform

Part I	PostgreSQL DDL, compiled from the live SQLAlchemy metadata — 306 lines
Part II	Declarative model sources, verbatim — 12 files, 963 lines
Part III	Alembic migrations, verbatim — 3 files, 409 lines
Coverage	13 tables · 10 native enum types

Generated PostgreSQL DDL

Compiled from the live metadata with `CreateTable` / `CreateIndex` against the PostgreSQL dialect, so this is exactly what the models ask the database for. Tables appear in dependency order; constraint names come from the convention in `base.py`. Enum types are emitted first because every table that uses one depends on it.

```
-- ============================================================
-- Enum types
-- ============================================================

CREATE TYPE access_status AS ENUM ('pending', 'approved', 'revoked');
CREATE TYPE difficulty AS ENUM ('easy', 'medium', 'hard');
CREATE TYPE exam_status AS ENUM ('draft', 'published', 'closed');
CREATE TYPE grade_status AS ENUM ('unanswered', 'auto_scored', 'pending_ai', 'ai_scored', 'examiner_reviewed');
CREATE TYPE login_access_status AS ENUM ('pending', 'approved', 'rejected');
CREATE TYPE proctor_event_type AS ENUM ('face_missing', 'multiple_faces', 'gaze_away', 'tab_switch', 'window_blur', 'fullscreen_exit', 'camera_blocked', 'paste_attempt', 'copy_attempt', 'devtools_open');
CREATE TYPE proctor_severity AS ENUM ('info', 'warning', 'critical');
CREATE TYPE question_type AS ENUM ('mcq', 'multi_select', 'short_answer', 'long_answer', 'image_upload');
CREATE TYPE session_status AS ENUM ('in_progress', 'submitted', 'auto_submitted', 'terminated');
CREATE TYPE user_role AS ENUM ('admin', 'examiner', 'candidate');

-- ============================================================
-- Tables, in dependency order
-- ============================================================

-- ------------------------------------------------------------
-- subjects
-- ------------------------------------------------------------
CREATE TABLE subjects (
 code VARCHAR(32) NOT NULL,
 name VARCHAR(150) NOT NULL,
 description TEXT,
 id UUID NOT NULL,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 CONSTRAINT pk_subjects PRIMARY KEY (id)
);
CREATE UNIQUE INDEX ix_subjects_code ON subjects (code);

-- ------------------------------------------------------------
-- users
-- ------------------------------------------------------------
CREATE TABLE users (
 email VARCHAR(255) NOT NULL,
 password_hash VARCHAR(255) NOT NULL,
 first_name VARCHAR(80) NOT NULL,
 last_name VARCHAR(80) NOT NULL,
 full_name VARCHAR(150) NOT NULL,
 role user_role NOT NULL,
 is_active BOOLEAN NOT NULL,
 access_status access_status NOT NULL,
 access_changed_at TIMESTAMP WITH TIME ZONE,
 access_changed_by_id UUID,
 access_note TEXT,
 last_login_at TIMESTAMP WITH TIME ZONE,
 id UUID NOT NULL,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 CONSTRAINT pk_users PRIMARY KEY (id),
 CONSTRAINT fk_users_access_changed_by_id_users FOREIGN KEY(access_changed_by_id) REFERENCES users (id) ON DELETE SET NULL
);
CREATE INDEX ix_users_access_status ON users (access_status);
CREATE UNIQUE INDEX ix_users_email ON users (email);
CREATE INDEX ix_users_role ON users (role);

-- ------------------------------------------------------------
-- exams
-- ------------------------------------------------------------
CREATE TABLE exams (
 subject_id UUID NOT NULL,
 title VARCHAR(200) NOT NULL,
 description TEXT,
```

```
  duration_minutes INTEGER NOT NULL,
  starts_at TIMESTAMP WITH TIME ZONE NOT NULL,
  ends_at TIMESTAMP WITH TIME ZONE NOT NULL,
  selection_rules JSONB NOT NULL,
  randomize BOOLEAN NOT NULL,
  shuffle_options BOOLEAN NOT NULL,
  negative_marking BOOLEAN NOT NULL,
  paper_salt VARCHAR(64) NOT NULL,
  proctor_config JSONB NOT NULL,
  grading_config JSONB NOT NULL,
  status exam_status NOT NULL,
  results_published BOOLEAN NOT NULL,
  created_by_id UUID,
  id UUID NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
  CONSTRAINT pk_exams PRIMARY KEY (id),
  CONSTRAINT fk_exams_subject_id_subjects FOREIGN KEY(subject_id) REFERENCES subjects (id) ON DELETE RESTRICT,
  CONSTRAINT fk_exams_created_by_id_users FOREIGN KEY(created_by_id) REFERENCES users (id) ON DELETE SET NULL
 );
 CREATE INDEX ix_exams_status ON exams (status);

 -- ----------------------------------------------------------------
 -- login_access_requests
 -- ----------------------------------------------------------------
 CREATE TABLE login_access_requests (
  candidate_id UUID NOT NULL,
  status login_access_status NOT NULL,
  requested_at TIMESTAMP WITH TIME ZONE NOT NULL,
  reviewed_at TIMESTAMP WITH TIME ZONE,
  reviewed_by_id UUID,
  review_note TEXT,
  id UUID NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
  CONSTRAINT pk_login_access_requests PRIMARY KEY (id),
  CONSTRAINT fk_login_access_requests_candidate_id_users FOREIGN KEY(candidate_id) REFERENCES users (id) ON DELETE
CASCADE,
  CONSTRAINT fk_login_access_requests_reviewed_by_id_users FOREIGN KEY(reviewed_by_id) REFERENCES users (id) ON DELETE
SET NULL
 );
 CREATE UNIQUE INDEX ix_login_access_requests_candidate_id ON login_access_requests (candidate_id);
 CREATE INDEX ix_login_access_requests_status ON login_access_requests (status);

 -- ----------------------------------------------------------------
 -- questions
 -- ----------------------------------------------------------------
 CREATE TABLE questions (
  subject_id UUID NOT NULL,
  question_type question_type NOT NULL,
  difficulty difficulty NOT NULL,
  body TEXT NOT NULL,
  model_answer TEXT,
  rubric JSONB,
  min_words INTEGER,
  max_words INTEGER,
  marks FLOAT NOT NULL,
  negative_marks FLOAT NOT NULL,
  tags JSONB,
  is_active BOOLEAN NOT NULL,
  created_by_id UUID,
  id UUID NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
  CONSTRAINT pk_questions PRIMARY KEY (id),
  CONSTRAINT fk_questions_subject_id_subjects FOREIGN KEY(subject_id) REFERENCES subjects (id) ON DELETE RESTRICT,
  CONSTRAINT fk_questions_created_by_id_users FOREIGN KEY(created_by_id) REFERENCES users (id) ON DELETE SET NULL
 );
 CREATE INDEX ix_questions_is_active ON questions (is_active);
 CREATE INDEX ix_questions_pool_lookup ON questions (subject_id, question_type, difficulty);

 -- ----------------------------------------------------------------
 -- exam_enrollments
 -- ----------------------------------------------------------------
 CREATE TABLE exam_enrollments (
  exam_id UUID NOT NULL,
  candidate_id UUID NOT NULL,
```

```
    assigned_by_id UUID,
    assigned_at TIMESTAMP WITH TIME ZONE NOT NULL,
    id UUID NOT NULL,
    CONSTRAINT pk_exam_enrollments PRIMARY KEY (id),
    CONSTRAINT uq_exam_enrollment UNIQUE (exam_id, candidate_id),
    CONSTRAINT fk_exam_enrollments_exam_id_exams FOREIGN KEY(exam_id) REFERENCES exams (id) ON DELETE CASCADE,
    CONSTRAINT fk_exam_enrollments_candidate_id_users FOREIGN KEY(candidate_id) REFERENCES users (id) ON DELETE CASCADE,
    CONSTRAINT fk_exam_enrollments_assigned_by_id_users FOREIGN KEY(assigned_by_id) REFERENCES users (id) ON DELETE SET
NULL
  );
  CREATE INDEX ix_exam_enrollments_candidate_id ON exam_enrollments (candidate_id);
  CREATE INDEX ix_exam_enrollments_exam_id ON exam_enrollments (exam_id);

  -- ----------------------------------------------------------------
  -- exam_questions
  -- ----------------------------------------------------------------
  CREATE TABLE exam_questions (
    exam_id UUID NOT NULL,
    question_id UUID NOT NULL,
    marks_override FLOAT,
    order_index INTEGER NOT NULL,
    id UUID NOT NULL,
    CONSTRAINT pk_exam_questions PRIMARY KEY (id),
    CONSTRAINT uq_exam_question UNIQUE (exam_id, question_id),
    CONSTRAINT fk_exam_questions_exam_id_exams FOREIGN KEY(exam_id) REFERENCES exams (id) ON DELETE CASCADE,
    CONSTRAINT fk_exam_questions_question_id_questions FOREIGN KEY(question_id) REFERENCES questions (id) ON DELETE
RESTRICT
  );
  CREATE INDEX ix_exam_questions_exam_id ON exam_questions (exam_id);

  -- ----------------------------------------------------------------
  -- exam_sessions
  -- ----------------------------------------------------------------
  CREATE TABLE exam_sessions (
    exam_id UUID NOT NULL,
    candidate_id UUID NOT NULL,
    paper_seed VARCHAR(64) NOT NULL,
    question_order JSONB NOT NULL,
    started_at TIMESTAMP WITH TIME ZONE NOT NULL,
    expires_at TIMESTAMP WITH TIME ZONE NOT NULL,
    submitted_at TIMESTAMP WITH TIME ZONE,
    last_heartbeat_at TIMESTAMP WITH TIME ZONE,
    status session_status NOT NULL,
    suspicion_score FLOAT NOT NULL,
    tab_switch_count INTEGER NOT NULL,
    is_flagged BOOLEAN NOT NULL,
    termination_reason TEXT,
    id UUID NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    CONSTRAINT pk_exam_sessions PRIMARY KEY (id),
    CONSTRAINT uq_exam_session_attempt UNIQUE (exam_id, candidate_id),
    CONSTRAINT fk_exam_sessions_exam_id_exams FOREIGN KEY(exam_id) REFERENCES exams (id) ON DELETE CASCADE,
    CONSTRAINT fk_exam_sessions_candidate_id_users FOREIGN KEY(candidate_id) REFERENCES users (id) ON DELETE CASCADE
  );
  CREATE INDEX ix_exam_sessions_candidate_id ON exam_sessions (candidate_id);
  CREATE INDEX ix_exam_sessions_exam_id ON exam_sessions (exam_id);
  CREATE INDEX ix_exam_sessions_is_flagged ON exam_sessions (is_flagged);
  CREATE INDEX ix_exam_sessions_status ON exam_sessions (status);

  -- ----------------------------------------------------------------
  -- question_options
  -- ----------------------------------------------------------------
  CREATE TABLE question_options (
    question_id UUID NOT NULL,
    text TEXT NOT NULL,
    is_correct BOOLEAN NOT NULL,
    order_index INTEGER NOT NULL,
    id UUID NOT NULL,
    CONSTRAINT pk_question_options PRIMARY KEY (id),
    CONSTRAINT fk_question_options_question_id_questions FOREIGN KEY(question_id) REFERENCES questions (id) ON DELETE
CASCADE
  );

  -- ----------------------------------------------------------------
  -- answers
  -- ----------------------------------------------------------------
```

```
CREATE TABLE answers (
 session_id UUID NOT NULL,
 question_id UUID NOT NULL,
 selected_option_ids JSONB,
 text_answer TEXT,
 image_object_key VARCHAR(512),
 image_thumb_key VARCHAR(512),
 ocr_text TEXT,
 ocr_confidence FLOAT,
 word_count INTEGER,
 awarded_marks FLOAT,
 max_marks FLOAT NOT NULL,
 grade_status grade_status NOT NULL,
 examiner_comment TEXT,
 graded_by_id UUID,
 graded_at TIMESTAMP WITH TIME ZONE,
 answered_at TIMESTAMP WITH TIME ZONE,
 id UUID NOT NULL,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 CONSTRAINT pk_answers PRIMARY KEY (id),
 CONSTRAINT uq_answer_per_question UNIQUE (session_id, question_id),
 CONSTRAINT fk_answers_session_id_exam_sessions FOREIGN KEY(session_id) REFERENCES exam_sessions (id) ON DELETE
CASCADE,
 CONSTRAINT fk_answers_question_id_questions FOREIGN KEY(question_id) REFERENCES questions (id) ON DELETE RESTRICT,
 CONSTRAINT fk_answers_graded_by_id_users FOREIGN KEY(graded_by_id) REFERENCES users (id) ON DELETE SET NULL
);
CREATE INDEX ix_answers_grade_status ON answers (grade_status);
CREATE INDEX ix_answers_session_id ON answers (session_id);

-- ---------------------------------------------------------------------
-- proctor_events
-- ---------------------------------------------------------------------
CREATE TABLE proctor_events (
 session_id UUID NOT NULL,
 event_type proctor_event_type NOT NULL,
 severity proctor_severity NOT NULL,
 occurred_at TIMESTAMP WITH TIME ZONE NOT NULL,
 server_received_at TIMESTAMP WITH TIME ZONE NOT NULL,
 duration_ms INTEGER,
 weight FLOAT NOT NULL,
 metadata JSONB,
 snapshot_object_key VARCHAR(512),
 id UUID NOT NULL,
 CONSTRAINT pk_proctor_events PRIMARY KEY (id),
 CONSTRAINT fk_proctor_events_session_id_exam_sessions FOREIGN KEY(session_id) REFERENCES exam_sessions (id) ON DELETE
CASCADE
);
CREATE INDEX ix_proctor_events_timeline ON proctor_events (session_id, occurred_at);

-- ---------------------------------------------------------------------
-- results
-- ---------------------------------------------------------------------
CREATE TABLE results (
 session_id UUID NOT NULL,
 total_marks FLOAT NOT NULL,
 obtained_marks FLOAT NOT NULL,
 percentage FLOAT NOT NULL,
 correct_count INTEGER NOT NULL,
 incorrect_count INTEGER NOT NULL,
 unanswered_count INTEGER NOT NULL,
 pending_review_count INTEGER NOT NULL,
 published BOOLEAN NOT NULL,
 published_at TIMESTAMP WITH TIME ZONE,
 id UUID NOT NULL,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 CONSTRAINT pk_results PRIMARY KEY (id),
 CONSTRAINT uq_results_session_id UNIQUE (session_id),
 CONSTRAINT fk_results_session_id_exam_sessions FOREIGN KEY(session_id) REFERENCES exam_sessions (id) ON DELETE CASCADE
);
CREATE INDEX ix_results_published ON results (published);

-- ---------------------------------------------------------------------
-- ai_evaluations
-- ---------------------------------------------------------------------
CREATE TABLE ai_evaluations (
```

```
 answer_id UUID NOT NULL,
 provider VARCHAR(50) NOT NULL,
 model VARCHAR(100),
 score FLOAT NOT NULL,
 max_score FLOAT NOT NULL,
 justification TEXT NOT NULL,
 confidence FLOAT NOT NULL,
 raw_response JSONB,
 error TEXT,
 id UUID NOT NULL,
 created_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 updated_at TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
 CONSTRAINT pk_ai_evaluations PRIMARY KEY (id),
 CONSTRAINT fk_ai_evaluations_answer_id_answers FOREIGN KEY(answer_id) REFERENCES answers (id) ON DELETE CASCADE
);
CREATE INDEX ix_ai_evaluations_answer_id ON ai_evaluations (answer_id);
```

II Declarative model sources

The SQLAlchemy 2.0 declarative definitions, verbatim. These are the authority: the DDL in Part I is derived from them, and Alembic diffs against them.

`base.py` — declarative base, naming convention, mixins

backend/app/db/base.py · 41 lines

```python
"""Declarative base, naming conventions and shared mixins."""

from __future__ import annotations

import uuid
from datetime import datetime

from sqlalchemy import DateTime, MetaData, func
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

# Explicit constraint naming so Alembic autogenerate produces stable, revertible names.
NAMING_CONVENTION = {
    "ix": "ix_%(column_0_label)s",
    "uq": "uq_%(table_name)s_%(column_0_name)s",
    "ck": "ck_%(table_name)s_%(constraint_name)s",
    "fk": "fk_%(table_name)s_%(column_0_name)s_%(referred_table_name)s",
    "pk": "pk_%(table_name)s",
}


class Base(DeclarativeBase):
    metadata = MetaData(naming_convention=NAMING_CONVENTION)


class UUIDPrimaryKeyMixin:
    id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), primary_key=True, default=uuid.uuid4
    )


class TimestampMixin:
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
        nullable=False,
    )
```

session.py — engine and session factory

backend/app/db/session.py · 34 lines

```python
"""Engine and session factory (sync SQLAlchemy 2.0 + psycopg 3)."""

from __future__ import annotations

from collections.abc import Iterator

from sqlalchemy import create_engine
from sqlalchemy.orm import Session, sessionmaker

from app.core.config import settings

engine = create_engine(
    settings.DATABASE_URL,
    pool_size=settings.DB_POOL_SIZE,
    max_overflow=settings.DB_MAX_OVERFLOW,
    pool_pre_ping=True,
    echo=settings.SQL_ECHO,
    future=True,
)

SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False, future=True)


def get_db() -> Iterator[Session]:
    """FastAPI dependency: one session per request, rolled back on error."""
    db = SessionLocal()
    try:
        yield db
        db.commit()
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

models/__init__.py — registry

backend/app/db/models/__init__.py · 57 lines

```python
"""Model package. Importing this registers every table on ``Base.metadata``."""

from app.db.base import Base
from app.db.models.enrollment import ExamEnrollment
from app.db.models.enums import (
    AccessStatus,
    Difficulty,
    ExamStatus,
    GradeStatus,
    LoginAccessStatus,
    ProctorEventType,
    ProctorSeverity,
    QuestionType,
    SessionStatus,
    UserRole,
)
from app.db.models.exam import (
    DEFAULT_GRADING_CONFIG,
    DEFAULT_PROCTOR_CONFIG,
    Exam,
    ExamQuestion,
)
from app.db.models.exam_session import Answer, ExamSession
from app.db.models.grading import AiEvaluation, Result
from app.db.models.login_access import LoginAccessRequest
from app.db.models.proctor import ProctorEvent
from app.db.models.question import Question, QuestionOption, Subject
from app.db.models.user import User

__all__ = [
    "DEFAULT_GRADING_CONFIG",
    "DEFAULT_PROCTOR_CONFIG",
    "AccessStatus",
    "AiEvaluation",
    "Answer",
    "Base",
    "Difficulty",
    "Exam",
    "ExamQuestion",
    "ExamEnrollment",
    "ExamSession",
    "ExamStatus",
    "GradeStatus",
    "LoginAccessRequest",
    "LoginAccessStatus",
    "ProctorEvent",
    "ProctorEventType",
    "ProctorSeverity",
    "Question",
    "QuestionOption",
    "QuestionType",
    "Result",
    "SessionStatus",
    "Subject",
    "User",
    "UserRole",
]
```

models/enums.py — domain enums

backend/app/db/models/enums.py · 99 lines

```python
"""Domain enums. These map to native PostgreSQL enum types."""

from __future__ import annotations

import enum


class UserRole(str, enum.Enum):
    ADMIN = "admin"
    EXAMINER = "examiner"
    CANDIDATE = "candidate"


class AccessStatus(str, enum.Enum):
    """Admin-controlled gate.

    Examiners register as ``PENDING`` and cannot touch the question bank or any exam
    until an admin flips them to ``APPROVED``. Candidates are approved on registration.
    """

    PENDING = "pending"
    APPROVED = "approved"
    REVOKED = "revoked"


class LoginAccessStatus(str, enum.Enum):
    """Permission to *use* the platform — deliberately separate from account state.

    A candidate's account is created active and stays active; this is the gate that an
    administrator (or an authorised examiner) opens. Revoking access sets an approved
    request back to ``REJECTED`` with a review note, so there is exactly one axis to
    reason about when deciding whether someone may log in.
    """

    PENDING = "pending"
    APPROVED = "approved"
    REJECTED = "rejected"


class QuestionType(str, enum.Enum):
    MCQ = "mcq"
    MULTI_SELECT = "multi_select"
    SHORT_ANSWER = "short_answer"
    LONG_ANSWER = "long_answer"
    IMAGE_UPLOAD = "image_upload"


OBJECTIVE_TYPES = {QuestionType.MCQ, QuestionType.MULTI_SELECT}
SUBJECTIVE_TYPES = {
    QuestionType.SHORT_ANSWER,
    QuestionType.LONG_ANSWER,
    QuestionType.IMAGE_UPLOAD,
}


class Difficulty(str, enum.Enum):
    EASY = "easy"
    MEDIUM = "medium"
    HARD = "hard"


class ExamStatus(str, enum.Enum):
    DRAFT = "draft"
    PUBLISHED = "published"
    CLOSED = "closed"


class SessionStatus(str, enum.Enum):
    IN_PROGRESS = "in_progress"
    SUBMITTED = "submitted"
    AUTO_SUBMITTED = "auto_submitted"
    TERMINATED = "terminated"
```

```python


class GradeStatus(str, enum.Enum):
    UNANSWERED = "unanswered"
    AUTO_SCORED = "auto_scored"
    PENDING_AI = "pending_ai"
    AI_SCORED = "ai_scored"
    EXAMINER_REVIEWED = "examiner_reviewed"


class ProctorEventType(str, enum.Enum):
    FACE_MISSING = "face_missing"
    MULTIPLE_FACES = "multiple_faces"
    GAZE_AWAY = "gaze_away"
    TAB_SWITCH = "tab_switch"
    WINDOW_BLUR = "window_blur"
    FULLSCREEN_EXIT = "fullscreen_exit"
    CAMERA_BLOCKED = "camera_blocked"
    PASTE_ATTEMPT = "paste_attempt"
    COPY_ATTEMPT = "copy_attempt"
    DEVTOOLS_OPEN = "devtools_open"


class ProctorSeverity(str, enum.Enum):
    INFO = "info"
    WARNING = "warning"
    CRITICAL = "critical"
```

models/user.py — User

backend/app/db/models/user.py · 95 lines

```python
"""Users and the admin-controlled access gate."""

from __future__ import annotations

import uuid
from datetime import datetime
from typing import TYPE_CHECKING

from sqlalchemy import Boolean, DateTime, Enum, ForeignKey, String, Text
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
from app.db.models.enums import AccessStatus, UserRole

if TYPE_CHECKING:
    from app.db.models.enrollment import ExamEnrollment
    from app.db.models.exam import Exam
    from app.db.models.exam_session import ExamSession
    from app.db.models.login_access import LoginAccessRequest


class User(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    __tablename__ = "users"

    email: Mapped[str] = mapped_column(String(255), unique=True, index=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    # first/last are the source of truth for greetings and sorting; full_name is kept
    # in sync so existing queries, exports and UI that read it keep working.
    first_name: Mapped[str] = mapped_column(String(80), nullable=False, default="")
    last_name: Mapped[str] = mapped_column(String(80), nullable=False, default="")
    full_name: Mapped[str] = mapped_column(String(150), nullable=False)
    role: Mapped[UserRole] = mapped_column(
        Enum(UserRole, name="user_role", values_callable=lambda e: [m.value for m in e]),
        nullable=False,
        index=True,
    )
    is_active: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)

    # --- Admin access gate ------------------------------------------------
    # An examiner is inert until an admin approves them. Candidates are approved
    # at registration; admins are approved by definition.
    access_status: Mapped[AccessStatus] = mapped_column(
        Enum(AccessStatus, name="access_status", values_callable=lambda e: [m.value for m in e]),
        default=AccessStatus.PENDING,
        nullable=False,
        index=True,
    )
    access_changed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    access_changed_by_id: Mapped[uuid.UUID | None] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
    )
    access_note: Mapped[str | None] = mapped_column(Text)

    last_login_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))

    access_changed_by: Mapped[User | None] = relationship(
        remote_side="User.id", foreign_keys=[access_changed_by_id]
    )
    exam_sessions: Mapped[list[ExamSession]] = relationship(
        back_populates="candidate",
        foreign_keys="ExamSession.candidate_id",
        cascade="all, delete-orphan",
    )
    exams_created: Mapped[list[Exam]] = relationship(
        back_populates="created_by", foreign_keys="Exam.created_by_id"
    )
    login_access_request: Mapped[LoginAccessRequest | None] = relationship(
        back_populates="candidate",
        foreign_keys="LoginAccessRequest.candidate_id",
        cascade="all, delete-orphan",
        uselist=False,
```

```python
    )
    enrollments: Mapped[list[ExamEnrollment]] = relationship(
        back_populates="candidate",
        foreign_keys="ExamEnrollment.candidate_id",
        cascade="all, delete-orphan",
    )

    @property
    def is_approved(self) -> bool:
        return self.access_status is AccessStatus.APPROVED

    @property
    def display_first_name(self) -> str:
        """First name for greetings, falling back to the first token of full_name."""
        return self.first_name or self.full_name.split(" ")[0]

    def set_name(self, first: str, last: str) -> None:
        self.first_name = first.strip()
        self.last_name = last.strip()
        self.full_name = " ".join(part for part in (self.first_name, self.last_name) if part)

    def __repr__(self) -> str:  # pragma: no cover - debugging aid
        return f"<User {self.email} role={self.role.value} access={self.access_status.value}>"
```

models/login_access.py — LoginAccessRequest

backend/app/db/models/login_access.py · 67 lines

```python
"""Login approval — permission to use the platform, distinct from account state.

A candidate's account is created **active** and stays active. What is gated is the *login*:
on their first sign-in attempt with valid credentials, a request is raised here and the
candidate is held on a pending screen until an administrator — or an examiner authorised
over them — decides.

Once approved, it stays approved: later logins reuse the same row and never queue again
unless someone explicitly revokes it (which moves the row back to ``REJECTED`` with a note).
"""

from __future__ import annotations

import uuid
from datetime import datetime
from typing import TYPE_CHECKING

from sqlalchemy import DateTime, Enum, ForeignKey, Text
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
from app.db.models.enums import LoginAccessStatus

if TYPE_CHECKING:
    from app.db.models.user import User


class LoginAccessRequest(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    __tablename__ = "login_access_requests"

    # One standing request per candidate: approval is a durable state, not an event log.
    candidate_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True),
        ForeignKey("users.id", ondelete="CASCADE"),
        nullable=False,
        unique=True,
        index=True,
    )
    status: Mapped[LoginAccessStatus] = mapped_column(
        Enum(
            LoginAccessStatus,
            name="login_access_status",
            values_callable=lambda e: [m.value for m in e],
        ),
        default=LoginAccessStatus.PENDING,
        nullable=False,
        index=True,
    )
    requested_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    reviewed_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    reviewed_by_id: Mapped[uuid.UUID | None] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
    )
    review_note: Mapped[str | None] = mapped_column(Text)

    candidate: Mapped[User] = relationship(
        back_populates="login_access_request", foreign_keys=[candidate_id]
    )
    reviewed_by: Mapped[User | None] = relationship(foreign_keys=[reviewed_by_id])

    @property
    def is_approved(self) -> bool:
        return self.status is LoginAccessStatus.APPROVED

    def __repr__(self) -> str:  # pragma: no cover — debugging aid
        return f"<LoginAccessRequest {self.candidate_id} {self.status.value}>"
```

models/enrollment.py — ExamEnrollment

backend/app/db/models/enrollment.py · 52 lines

```python
"""Exam enrolment — which candidates a given exam is actually assigned to.

Approved login is permission to use the platform; it is *not* permission to sit every
paper on it. A candidate sees and can start only the exams they are enrolled in, and this
table is also what defines an examiner's authority over a candidate: an examiner may
review a candidate's login request only if that candidate is enrolled in one of the
examiner's own exams.
"""

from __future__ import annotations

import uuid
from datetime import datetime
from typing import TYPE_CHECKING

from sqlalchemy import DateTime, ForeignKey, UniqueConstraint
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, UUIDPrimaryKeyMixin

if TYPE_CHECKING:
    from app.db.models.exam import Exam
    from app.db.models.user import User


class ExamEnrollment(UUIDPrimaryKeyMixin, Base):
    __tablename__ = "exam_enrollments"
    __table_args__ = (UniqueConstraint("exam_id", "candidate_id", name="uq_exam_enrollment"),)

    exam_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True),
        ForeignKey("exams.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )
    candidate_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True),
        ForeignKey("users.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )
    assigned_by_id: Mapped[uuid.UUID | None] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
    )
    assigned_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)

    exam: Mapped[Exam] = relationship(back_populates="enrollments")
    candidate: Mapped[User] = relationship(
        back_populates="enrollments", foreign_keys=[candidate_id]
    )
    assigned_by: Mapped[User | None] = relationship(foreign_keys=[assigned_by_id])
```

models/question.py — Subject, Question, QuestionOption

backend/app/db/models/question.py · 88 lines

```python
"""Subjects, question bank and options."""

from __future__ import annotations

import uuid
from typing import TYPE_CHECKING, Any

from sqlalchemy import Boolean, Enum, Float, ForeignKey, Index, Integer, String, Text
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
from app.db.models.enums import Difficulty, QuestionType

if TYPE_CHECKING:
    from app.db.models.user import User


class Subject(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    __tablename__ = "subjects"

    code: Mapped[str] = mapped_column(String(32), unique=True, index=True, nullable=False)
    name: Mapped[str] = mapped_column(String(150), nullable=False)
    description: Mapped[str | None] = mapped_column(Text)

    questions: Mapped[list[Question]] = relationship(back_populates="subject")


class Question(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    __tablename__ = "questions"
    __table_args__ = (
        Index("ix_questions_pool_lookup", "subject_id", "question_type", "difficulty"),
    )

    subject_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("subjects.id", ondelete="RESTRICT"), nullable=False
    )
    question_type: Mapped[QuestionType] = mapped_column(
        Enum(QuestionType, name="question_type", values_callable=lambda e: [m.value for m in e]),
        nullable=False,
    )
    difficulty: Mapped[Difficulty] = mapped_column(
        Enum(Difficulty, name="difficulty", values_callable=lambda e: [m.value for m in e]),
        nullable=False,
        default=Difficulty.MEDIUM,
    )
    body: Mapped[str] = mapped_column(Text, nullable=False)
    # Reference answer for subjective types; also shown to the examiner while grading.
    model_answer: Mapped[str | None] = mapped_column(Text)
    # Free-form rubric handed to the grader, e.g. {"criteria": [{"name":..,"weight":..}]}
    rubric: Mapped[dict[str, Any] | None] = mapped_column(JSONB)
    # Word bounds for written answers. Enforced server-side on every autosave, and shown
    # as a live counter in the runner so a candidate is never surprised at submit time.
    min_words: Mapped[int | None] = mapped_column(Integer)
    max_words: Mapped[int | None] = mapped_column(Integer)
    marks: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
    negative_marks: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    tags: Mapped[list[str] | None] = mapped_column(JSONB)
    is_active: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False, index=True)

    created_by_id: Mapped[uuid.UUID | None] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
    )

    subject: Mapped[Subject] = relationship(back_populates="questions")
    created_by: Mapped[User | None] = relationship()
    options: Mapped[list[QuestionOption]] = relationship(
        back_populates="question",
        cascade="all, delete-orphan",
        order_by="QuestionOption.order_index",
    )
```

```python

    def __repr__(self) -> str:  # pragma: no cover — debugging aid
        return f"<Question {self.question_type.value} {self.body[:40]!r}>"


class QuestionOption(UUIDPrimaryKeyMixin, Base):
    __tablename__ = "question_options"

    question_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("questions.id", ondelete="CASCADE"), nullable=False
    )
    text: Mapped[str] = mapped_column(Text, nullable=False)
    is_correct: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
    order_index: Mapped[int] = mapped_column(Integer, default=0, nullable=False)

    question: Mapped[Question] = relationship(back_populates="options")
```

models/exam.py — Exam, ExamQuestion

backend/app/db/models/exam.py · 145 lines

```python
"""Exams, their configuration, and the question pool bound to each exam."""

from __future__ import annotations

import uuid
from datetime import datetime
from typing import TYPE_CHECKING, Any

from sqlalchemy import (
    Boolean,
    DateTime,
    Enum,
    Float,
    ForeignKey,
    Integer,
    String,
    Text,
    UniqueConstraint,
)
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
from app.db.models.enums import ExamStatus

if TYPE_CHECKING:
    from app.db.models.enrollment import ExamEnrollment
    from app.db.models.exam_session import ExamSession
    from app.db.models.question import Question, Subject
    from app.db.models.user import User


DEFAULT_PROCTOR_CONFIG: dict[str, Any] = {
    "webcam_enabled": True,
    "gaze_tracking_enabled": True,
    "gaze_sensitivity": 0.6,  # 0..1, higher = stricter
    "max_tab_switches": 3,  # warnings before the session is auto-flagged
    "terminate_on_score": 100.0,  # suspicion score that ends the session
    "flag_on_score": 45.0,  # suspicion score that flags for review
    "snapshot_interval_seconds": 60,
    "require_fullscreen": True,
    "block_copy_paste": True,
    "weights": {
        "face_missing": 6.0,
        "multiple_faces": 15.0,
        "gaze_away": 3.0,
        "tab_switch": 10.0,
        "window_blur": 4.0,
        "fullscreen_exit": 8.0,
        "camera_blocked": 20.0,
        "paste_attempt": 12.0,
        "copy_attempt": 6.0,
        "devtools_open": 25.0,
    },
}

DEFAULT_GRADING_CONFIG: dict[str, Any] = {
    "partial_credit_multi_select": False,
    "auto_publish_results": False,
    "grader_provider": None,  # None -> fall back to the app-level setting
    "grader_model": None,
}


class Exam(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    __tablename__ = "exams"

    subject_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("subjects.id", ondelete="RESTRICT"), nullable=False
    )
    title: Mapped[str] = mapped_column(String(200), nullable=False)
```

```python
    description: Mapped[str | None] = mapped_column(Text)

    duration_minutes: Mapped[int] = mapped_column(Integer, nullable=False)
    starts_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    ends_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)

    # {"rules": [{"question_type": "mcq", "difficulty": "easy", "count": 10}, ...]}
    selection_rules: Mapped[dict[str, Any]] = mapped_column(JSONB, nullable=False, default=dict)
    randomize: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
    shuffle_options: Mapped[bool] = mapped_column(Boolean, default=True, nullable=False)
    negative_marking: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)
    # Stable per-exam salt so the deterministic paper seed cannot be guessed from ids alone.
    paper_salt: Mapped[str] = mapped_column(String(64), nullable=False)

    proctor_config: Mapped[dict[str, Any]] = mapped_column(
        JSONB, nullable=False, default=lambda: dict(DEFAULT_PROCTOR_CONFIG)
    )
    grading_config: Mapped[dict[str, Any]] = mapped_column(
        JSONB, nullable=False, default=lambda: dict(DEFAULT_GRADING_CONFIG)
    )

    status: Mapped[ExamStatus] = mapped_column(
        Enum(ExamStatus, name="exam_status", values_callable=lambda e: [m.value for m in e]),
        default=ExamStatus.DRAFT,
        nullable=False,
        index=True,
    )
    results_published: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False)

    created_by_id: Mapped[uuid.UUID | None] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
    )

    subject: Mapped[Subject] = relationship()
    created_by: Mapped[User | None] = relationship(
        back_populates="exams_created", foreign_keys=[created_by_id]
    )
    exam_questions: Mapped[list[ExamQuestion]] = relationship(
        back_populates="exam", cascade="all, delete-orphan"
    )
    sessions: Mapped[list[ExamSession]] = relationship(
        back_populates="exam", cascade="all, delete-orphan"
    )
    enrollments: Mapped[list[ExamEnrollment]] = relationship(
        back_populates="exam", cascade="all, delete-orphan"
    )

    @property
    def total_marks(self) -> float:
        return sum(eq.effective_marks for eq in self.exam_questions)


class ExamQuestion(UUIDPrimaryKeyMixin, Base):
    """The pool an exam draws from. A candidate's paper is a subset of these."""

    __tablename__ = "exam_questions"
    __table_args__ = (UniqueConstraint("exam_id", "question_id", name="uq_exam_question"),)

    exam_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("exams.id", ondelete="CASCADE"), nullable=False, index=True
    )
    question_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("questions.id", ondelete="RESTRICT"), nullable=False
    )
    marks_override: Mapped[float | None] = mapped_column(Float)
    order_index: Mapped[int] = mapped_column(Integer, default=0, nullable=False)

    exam: Mapped[Exam] = relationship(back_populates="exam_questions")
    question: Mapped[Question] = relationship()

    @property
    def effective_marks(self) -> float:
        return self.marks_override if self.marks_override is not None else self.question.marks
```

models/exam_session.py — ExamSession, Answer

backend/app/db/models/exam_session.py · 150 lines

```python
"""Live exam sessions and the answers recorded inside them."""

from __future__ import annotations

import uuid
from datetime import datetime
from typing import TYPE_CHECKING, Any

from sqlalchemy import (
    DateTime,
    Enum,
    Float,
    ForeignKey,
    Integer,
    String,
    Text,
    UniqueConstraint,
)
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin
from app.db.models.enums import GradeStatus, SessionStatus

if TYPE_CHECKING:
    from app.db.models.exam import Exam
    from app.db.models.grading import AiEvaluation, Result
    from app.db.models.proctor import ProctorEvent
    from app.db.models.question import Question
    from app.db.models.user import User


class ExamSession(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    """One candidate's attempt at one exam.

    The unique (exam_id, candidate_id) constraint is what enforces a single attempt.
    ``question_order`` is persisted at start so a server restart mid-exam cannot reshuffle
    the paper under the candidate.
    """

    __tablename__ = "exam_sessions"
    __table_args__ = (UniqueConstraint("exam_id", "candidate_id", name="uq_exam_session_attempt"),)

    exam_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("exams.id", ondelete="CASCADE"), nullable=False, index=True
    )
    candidate_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="CASCADE"), nullable=False, index=True
    )

    paper_seed: Mapped[str] = mapped_column(String(64), nullable=False)
    # [{"question_id": "...", "option_order": ["...", "..."]}, ...]
    question_order: Mapped[list[dict[str, Any]]] = mapped_column(JSONB, nullable=False)

    started_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    expires_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    submitted_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    last_heartbeat_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))

    status: Mapped[SessionStatus] = mapped_column(
        Enum(SessionStatus, name="session_status", values_callable=lambda e: [m.value for m in e]),
        default=SessionStatus.IN_PROGRESS,
        nullable=False,
        index=True,
    )

    suspicion_score: Mapped[float] = mapped_column(Float, default=0.0, nullable=False)
    tab_switch_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    is_flagged: Mapped[bool] = mapped_column(default=False, nullable=False, index=True)
    termination_reason: Mapped[str | None] = mapped_column(Text)

```

```python
    exam: Mapped[Exam] = relationship(back_populates="sessions")
    candidate: Mapped[User] = relationship(
        back_populates="exam_sessions", foreign_keys=[candidate_id]
    )
    answers: Mapped[list[Answer]] = relationship(
        back_populates="session", cascade="all, delete-orphan"
    )
    proctor_events: Mapped[list[ProctorEvent]] = relationship(
        back_populates="session", cascade="all, delete-orphan"
    )
    result: Mapped[Result | None] = relationship(
        back_populates="session", cascade="all, delete-orphan", uselist=False
    )

    @property
    def is_open(self) -> bool:
        return self.status is SessionStatus.IN_PROGRESS


class Answer(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    __tablename__ = "answers"
    __table_args__ = (UniqueConstraint("session_id", "question_id", name="uq_answer_per_question"),)

    session_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True),
        ForeignKey("exam_sessions.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )
    question_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("questions.id", ondelete="RESTRICT"), nullable=False
    )

    # Objective answers: list of chosen option ids (stringified UUIDs).
    selected_option_ids: Mapped[list[str] | None] = mapped_column(JSONB)
    # Subjective answers: free text, or an object key for a handwritten scan.
    text_answer: Mapped[str | None] = mapped_column(Text)
    image_object_key: Mapped[str | None] = mapped_column(String(512))
    # Server-generated so the examiner queue can render a grid of scans without pulling
    # a dozen multi-megabyte originals through presigned URLs.
    image_thumb_key: Mapped[str | None] = mapped_column(String(512))
    # OCR pre-pass over a handwritten scan. Advisory only - the examiner reads the image.
    ocr_text: Mapped[str | None] = mapped_column(Text)
    ocr_confidence: Mapped[float | None] = mapped_column(Float)
    # Recorded at save time so grading and analytics never re-tokenise the text.
    word_count: Mapped[int | None] = mapped_column(Integer)

    awarded_marks: Mapped[float | None] = mapped_column(Float)
    max_marks: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
    grade_status: Mapped[GradeStatus] = mapped_column(
        Enum(GradeStatus, name="grade_status", values_callable=lambda e: [m.value for m in e]),
        default=GradeStatus.UNANSWERED,
        nullable=False,
        index=True,
    )
    examiner_comment: Mapped[str | None] = mapped_column(Text)
    graded_by_id: Mapped[uuid.UUID | None] = mapped_column(
        PGUUID(as_uuid=True), ForeignKey("users.id", ondelete="SET NULL")
    )
    graded_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))
    answered_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))

    session: Mapped[ExamSession] = relationship(back_populates="answers")
    question: Mapped[Question] = relationship()
    graded_by: Mapped[User | None] = relationship(foreign_keys=[graded_by_id])
    ai_evaluations: Mapped[list[AiEvaluation]] = relationship(
        back_populates="answer",
        cascade="all, delete-orphan",
        order_by="AiEvaluation.created_at.desc()",
    )

    @property
    def latest_ai_evaluation(self) -> AiEvaluation | None:
        return self.ai_evaluations[0] if self.ai_evaluations else None

    @property
```

```
    def is_answered(self) -> bool:
        return bool(self.selected_option_ids or self.text_answer or self.image_object_key)
```

models/proctor.py — ProctorEvent

backend/app/db/models/proctor.py · 56 lines

```python
"""Proctoring evidence: one row per behavioural or vision signal."""

from __future__ import annotations

import uuid
from datetime import datetime
from typing import TYPE_CHECKING, Any

from sqlalchemy import DateTime, Enum, Float, ForeignKey, Index, String
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, UUIDPrimaryKeyMixin
from app.db.models.enums import ProctorEventType, ProctorSeverity

if TYPE_CHECKING:
    from app.db.models.exam_session import ExamSession


class ProctorEvent(UUIDPrimaryKeyMixin, Base):
    __tablename__ = "proctor_events"
    __table_args__ = (Index("ix_proctor_events_timeline", "session_id", "occurred_at"),)

    session_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True),
        ForeignKey("exam_sessions.id", ondelete="CASCADE"),
        nullable=False,
    )
    event_type: Mapped[ProctorEventType] = mapped_column(
        Enum(
            ProctorEventType,
            name="proctor_event_type",
            values_callable=lambda e: [m.value for m in e],
        ),
        nullable=False,
    )
    severity: Mapped[ProctorSeverity] = mapped_column(
        Enum(
            ProctorSeverity,
            name="proctor_severity",
            values_callable=lambda e: [m.value for m in e],
        ),
        default=ProctorSeverity.INFO,
        nullable=False,
    )
    # Client clock, echoed for ordering inside a batch; server_received_at is authoritative.
    occurred_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    server_received_at: Mapped[datetime] = mapped_column(DateTime(timezone=True), nullable=False)
    duration_ms: Mapped[int | None] = mapped_column()
    weight: Mapped[float] = mapped_column(Float, default=0.0, nullable=False)
    # e.g. {"face_count": 2, "yaw": -34.2, "confidence": 0.91}
    event_metadata: Mapped[dict[str, Any] | None] = mapped_column("metadata", JSONB)
    snapshot_object_key: Mapped[str | None] = mapped_column(String(512))

    session: Mapped[ExamSession] = relationship(back_populates="proctor_events")
```

models/grading.py — AiEvaluation, Result

backend/app/db/models/grading.py · 67 lines

```python
"""Machine evaluations of subjective answers, and final per-session results."""

from __future__ import annotations

import uuid
from datetime import datetime
from typing import TYPE_CHECKING, Any

from sqlalchemy import Boolean, DateTime, Float, ForeignKey, Integer, String, Text
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.dialects.postgresql import UUID as PGUUID
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.db.base import Base, TimestampMixin, UUIDPrimaryKeyMixin

if TYPE_CHECKING:
    from app.db.models.exam_session import Answer, ExamSession


class AiEvaluation(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    """First-pass score produced by the configured grader.

    Never authoritative on its own - an examiner reviews and may override it. Kept as a
    history (one row per grading run) so a re-grade does not destroy the earlier verdict.
    """

    __tablename__ = "ai_evaluations"

    answer_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True),
        ForeignKey("answers.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
    )
    provider: Mapped[str] = mapped_column(String(50), nullable=False)
    model: Mapped[str | None] = mapped_column(String(100))
    score: Mapped[float] = mapped_column(Float, nullable=False)
    max_score: Mapped[float] = mapped_column(Float, nullable=False)
    justification: Mapped[str] = mapped_column(Text, nullable=False)
    confidence: Mapped[float] = mapped_column(Float, default=0.0, nullable=False)
    raw_response: Mapped[dict[str, Any] | None] = mapped_column(JSONB)
    error: Mapped[str | None] = mapped_column(Text)

    answer: Mapped[Answer] = relationship(back_populates="ai_evaluations")


class Result(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    __tablename__ = "results"

    session_id: Mapped[uuid.UUID] = mapped_column(
        PGUUID(as_uuid=True),
        ForeignKey("exam_sessions.id", ondelete="CASCADE"),
        nullable=False,
        unique=True,
    )
    total_marks: Mapped[float] = mapped_column(Float, nullable=False)
    obtained_marks: Mapped[float] = mapped_column(Float, nullable=False)
    percentage: Mapped[float] = mapped_column(Float, nullable=False)
    correct_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    incorrect_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    unanswered_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    pending_review_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)

    published: Mapped[bool] = mapped_column(Boolean, default=False, nullable=False, index=True)
    published_at: Mapped[datetime | None] = mapped_column(DateTime(timezone=True))

    session: Mapped[ExamSession] = relationship(back_populates="result")
```

III Alembic migrations

Linear history. Each revision carries both `upgrade()` and `downgrade()`.

03ad33e6bd09_initial_schema.py

backend/alembic/versions/03ad33e6bd09_initial_schema.py · 253 lines

```python
"""initial schema

Revision ID: 03ad33e6bd09
Revises:
Create Date: 2026-08-27 12:36:58.989165
"""
from __future__ import annotations

from collections.abc import Sequence

import sqlalchemy as sa
from alembic import op
from sqlalchemy.dialects import postgresql
revision: str = '03ad33e6bd09'
down_revision: str | None = None
branch_labels: str | Sequence[str] | None = None
depends_on: str | Sequence[str] | None = None


def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('subjects',
    sa.Column('code', sa.String(length=32), nullable=False),
    sa.Column('name', sa.String(length=150), nullable=False),
    sa.Column('description', sa.Text(), nullable=True),
    sa.Column('id', sa.UUID(), nullable=False),
    sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.PrimaryKeyConstraint('id', name=op.f('pk_subjects'))
    )
    op.create_index(op.f('ix_subjects_code'), 'subjects', ['code'], unique=True)
    op.create_table('users',
    sa.Column('email', sa.String(length=255), nullable=False),
    sa.Column('password_hash', sa.String(length=255), nullable=False),
    sa.Column('full_name', sa.String(length=150), nullable=False),
    sa.Column('role', sa.Enum('admin', 'examiner', 'candidate', name='user_role'), nullable=False),
    sa.Column('is_active', sa.Boolean(), nullable=False),
    sa.Column('access_status', sa.Enum('pending', 'approved', 'revoked', name='access_status'), nullable=False),
    sa.Column('access_changed_at', sa.DateTime(timezone=True), nullable=True),
    sa.Column('access_changed_by_id', sa.UUID(), nullable=True),
    sa.Column('access_note', sa.Text(), nullable=True),
    sa.Column('last_login_at', sa.DateTime(timezone=True), nullable=True),
    sa.Column('id', sa.UUID(), nullable=False),
    sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.ForeignKeyConstraint(['access_changed_by_id'], ['users.id'], name=op.f('fk_users_access_changed_by_id_users'),
ondelete='SET NULL'),
    sa.PrimaryKeyConstraint('id', name=op.f('pk_users'))
    )
    op.create_index(op.f('ix_users_access_status'), 'users', ['access_status'], unique=False)
    op.create_index(op.f('ix_users_email'), 'users', ['email'], unique=True)
    op.create_index(op.f('ix_users_role'), 'users', ['role'], unique=False)
    op.create_table('exams',
    sa.Column('subject_id', sa.UUID(), nullable=False),
    sa.Column('title', sa.String(length=200), nullable=False),
    sa.Column('description', sa.Text(), nullable=True),
    sa.Column('duration_minutes', sa.Integer(), nullable=False),
    sa.Column('starts_at', sa.DateTime(timezone=True), nullable=False),
    sa.Column('ends_at', sa.DateTime(timezone=True), nullable=False),
    sa.Column('selection_rules', postgresql.JSONB(astext_type=sa.Text()), nullable=False),
    sa.Column('randomize', sa.Boolean(), nullable=False),
    sa.Column('shuffle_options', sa.Boolean(), nullable=False),
    sa.Column('negative_marking', sa.Boolean(), nullable=False),
    sa.Column('paper_salt', sa.String(length=64), nullable=False),
    sa.Column('proctor_config', postgresql.JSONB(astext_type=sa.Text()), nullable=False),
    sa.Column('grading_config', postgresql.JSONB(astext_type=sa.Text()), nullable=False),
    sa.Column('status', sa.Enum('draft', 'published', 'closed', name='exam_status'), nullable=False),
```

```
        sa.Column('results_published', sa.Boolean(), nullable=False),
        sa.Column('created_by_id', sa.UUID(), nullable=True),
        sa.Column('id', sa.UUID(), nullable=False),
        sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.ForeignKeyConstraint(['created_by_id'], ['users.id'], name=op.f('fk_exams_created_by_id_users'), ondelete='SET
NULL'),
        sa.ForeignKeyConstraint(['subject_id'], ['subjects.id'], name=op.f('fk_exams_subject_id_subjects'),
ondelete='RESTRICT'),
        sa.PrimaryKeyConstraint('id', name=op.f('pk_exams'))
        )
        op.create_index(op.f('ix_exams_status'), 'exams', ['status'], unique=False)
        op.create_table('questions',
        sa.Column('subject_id', sa.UUID(), nullable=False),
        sa.Column('question_type', sa.Enum('mcq', 'multi_select', 'short_answer', 'long_answer', 'image_upload',
name='question_type'), nullable=False),
        sa.Column('difficulty', sa.Enum('easy', 'medium', 'hard', name='difficulty'), nullable=False),
        sa.Column('body', sa.Text(), nullable=False),
        sa.Column('model_answer', sa.Text(), nullable=True),
        sa.Column('rubric', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
        sa.Column('marks', sa.Float(), nullable=False),
        sa.Column('negative_marks', sa.Float(), nullable=False),
        sa.Column('tags', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
        sa.Column('is_active', sa.Boolean(), nullable=False),
        sa.Column('created_by_id', sa.UUID(), nullable=True),
        sa.Column('id', sa.UUID(), nullable=False),
        sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.ForeignKeyConstraint(['created_by_id'], ['users.id'], name=op.f('fk_questions_created_by_id_users'),
ondelete='SET NULL'),
        sa.ForeignKeyConstraint(['subject_id'], ['subjects.id'], name=op.f('fk_questions_subject_id_subjects'),
ondelete='RESTRICT'),
        sa.PrimaryKeyConstraint('id', name=op.f('pk_questions'))
        )
        op.create_index(op.f('ix_questions_is_active'), 'questions', ['is_active'], unique=False)
        op.create_index('ix_questions_pool_lookup', 'questions', ['subject_id', 'question_type', 'difficulty'],
unique=False)
        op.create_table('exam_questions',
        sa.Column('exam_id', sa.UUID(), nullable=False),
        sa.Column('question_id', sa.UUID(), nullable=False),
        sa.Column('marks_override', sa.Float(), nullable=True),
        sa.Column('order_index', sa.Integer(), nullable=False),
        sa.Column('id', sa.UUID(), nullable=False),
        sa.ForeignKeyConstraint(['exam_id'], ['exams.id'], name=op.f('fk_exam_questions_exam_id_exams'),
ondelete='CASCADE'),
        sa.ForeignKeyConstraint(['question_id'], ['questions.id'], name=op.f('fk_exam_questions_question_id_questions'),
ondelete='RESTRICT'),
        sa.PrimaryKeyConstraint('id', name=op.f('pk_exam_questions')),
        sa.UniqueConstraint('exam_id', 'question_id', name='uq_exam_question')
        )
        op.create_index(op.f('ix_exam_questions_exam_id'), 'exam_questions', ['exam_id'], unique=False)
        op.create_table('exam_sessions',
        sa.Column('exam_id', sa.UUID(), nullable=False),
        sa.Column('candidate_id', sa.UUID(), nullable=False),
        sa.Column('paper_seed', sa.String(length=64), nullable=False),
        sa.Column('question_order', postgresql.JSONB(astext_type=sa.Text()), nullable=False),
        sa.Column('started_at', sa.DateTime(timezone=True), nullable=False),
        sa.Column('expires_at', sa.DateTime(timezone=True), nullable=False),
        sa.Column('submitted_at', sa.DateTime(timezone=True), nullable=True),
        sa.Column('last_heartbeat_at', sa.DateTime(timezone=True), nullable=True),
        sa.Column('status', sa.Enum('in_progress', 'submitted', 'auto_submitted', 'terminated', name='session_status'),
nullable=False),
        sa.Column('suspicion_score', sa.Float(), nullable=False),
        sa.Column('tab_switch_count', sa.Integer(), nullable=False),
        sa.Column('is_flagged', sa.Boolean(), nullable=False),
        sa.Column('termination_reason', sa.Text(), nullable=True),
        sa.Column('id', sa.UUID(), nullable=False),
        sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.ForeignKeyConstraint(['candidate_id'], ['users.id'], name=op.f('fk_exam_sessions_candidate_id_users'),
ondelete='CASCADE'),
        sa.ForeignKeyConstraint(['exam_id'], ['exams.id'], name=op.f('fk_exam_sessions_exam_id_exams'),
ondelete='CASCADE'),
        sa.PrimaryKeyConstraint('id', name=op.f('pk_exam_sessions')),
        sa.UniqueConstraint('exam_id', 'candidate_id', name='uq_exam_session_attempt')
        )
        op.create_index(op.f('ix_exam_sessions_candidate_id'), 'exam_sessions', ['candidate_id'], unique=False)
```

```
        op.create_index(op.f('ix_exam_sessions_exam_id'), 'exam_sessions', ['exam_id'], unique=False)
        op.create_index(op.f('ix_exam_sessions_is_flagged'), 'exam_sessions', ['is_flagged'], unique=False)
        op.create_index(op.f('ix_exam_sessions_status'), 'exam_sessions', ['status'], unique=False)
        op.create_table('question_options',
        sa.Column('question_id', sa.UUID(), nullable=False),
        sa.Column('text', sa.Text(), nullable=False),
        sa.Column('is_correct', sa.Boolean(), nullable=False),
        sa.Column('order_index', sa.Integer(), nullable=False),
        sa.Column('id', sa.UUID(), nullable=False),
        sa.ForeignKeyConstraint(['question_id'], ['questions.id'], name=op.f('fk_question_options_question_id_questions'),
ondelete='CASCADE'),
        sa.PrimaryKeyConstraint('id', name=op.f('pk_question_options'))
        )
        op.create_table('answers',
        sa.Column('session_id', sa.UUID(), nullable=False),
        sa.Column('question_id', sa.UUID(), nullable=False),
        sa.Column('selected_option_ids', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
        sa.Column('text_answer', sa.Text(), nullable=True),
        sa.Column('image_object_key', sa.String(length=512), nullable=True),
        sa.Column('awarded_marks', sa.Float(), nullable=True),
        sa.Column('max_marks', sa.Float(), nullable=False),
        sa.Column('grade_status', sa.Enum('unanswered', 'auto_scored', 'pending_ai', 'ai_scored', 'examiner_reviewed',
name='grade_status'), nullable=False),
        sa.Column('examiner_comment', sa.Text(), nullable=True),
        sa.Column('graded_by_id', sa.UUID(), nullable=True),
        sa.Column('graded_at', sa.DateTime(timezone=True), nullable=True),
        sa.Column('answered_at', sa.DateTime(timezone=True), nullable=True),
        sa.Column('id', sa.UUID(), nullable=False),
        sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.ForeignKeyConstraint(['graded_by_id'], ['users.id'], name=op.f('fk_answers_graded_by_id_users'), ondelete='SET
NULL'),
        sa.ForeignKeyConstraint(['question_id'], ['questions.id'], name=op.f('fk_answers_question_id_questions'),
ondelete='RESTRICT'),
        sa.ForeignKeyConstraint(['session_id'], ['exam_sessions.id'], name=op.f('fk_answers_session_id_exam_sessions'),
ondelete='CASCADE'),
        sa.PrimaryKeyConstraint('id', name=op.f('pk_answers')),
        sa.UniqueConstraint('session_id', 'question_id', name='uq_answer_per_question')
        )
        op.create_index(op.f('ix_answers_grade_status'), 'answers', ['grade_status'], unique=False)
        op.create_index(op.f('ix_answers_session_id'), 'answers', ['session_id'], unique=False)
        op.create_table('proctor_events',
        sa.Column('session_id', sa.UUID(), nullable=False),
        sa.Column('event_type', sa.Enum('face_missing', 'multiple_faces', 'gaze_away', 'tab_switch', 'window_blur',
'fullscreen_exit', 'camera_blocked', 'paste_attempt', 'copy_attempt', 'devtools_open', name='proctor_event_type'),
nullable=False),
        sa.Column('severity', sa.Enum('info', 'warning', 'critical', name='proctor_severity'), nullable=False),
        sa.Column('occurred_at', sa.DateTime(timezone=True), nullable=False),
        sa.Column('server_received_at', sa.DateTime(timezone=True), nullable=False),
        sa.Column('duration_ms', sa.Integer(), nullable=True),
        sa.Column('weight', sa.Float(), nullable=False),
        sa.Column('metadata', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
        sa.Column('snapshot_object_key', sa.String(length=512), nullable=True),
        sa.Column('id', sa.UUID(), nullable=False),
        sa.ForeignKeyConstraint(['session_id'], ['exam_sessions.id'],
name=op.f('fk_proctor_events_session_id_exam_sessions'), ondelete='CASCADE'),
        sa.PrimaryKeyConstraint('id', name=op.f('pk_proctor_events'))
        )
        op.create_index('ix_proctor_events_timeline', 'proctor_events', ['session_id', 'occurred_at'], unique=False)
        op.create_table('results',
        sa.Column('session_id', sa.UUID(), nullable=False),
        sa.Column('total_marks', sa.Float(), nullable=False),
        sa.Column('obtained_marks', sa.Float(), nullable=False),
        sa.Column('percentage', sa.Float(), nullable=False),
        sa.Column('correct_count', sa.Integer(), nullable=False),
        sa.Column('incorrect_count', sa.Integer(), nullable=False),
        sa.Column('unanswered_count', sa.Integer(), nullable=False),
        sa.Column('pending_review_count', sa.Integer(), nullable=False),
        sa.Column('published', sa.Boolean(), nullable=False),
        sa.Column('published_at', sa.DateTime(timezone=True), nullable=True),
        sa.Column('id', sa.UUID(), nullable=False),
        sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
        sa.ForeignKeyConstraint(['session_id'], ['exam_sessions.id'], name=op.f('fk_results_session_id_exam_sessions'),
ondelete='CASCADE'),
        sa.PrimaryKeyConstraint('id', name=op.f('pk_results')),
        sa.UniqueConstraint('session_id', name=op.f('uq_results_session_id'))
```

```
    )
    op.create_index(op.f('ix_results_published'), 'results', ['published'], unique=False)
    op.create_table('ai_evaluations',
    sa.Column('answer_id', sa.UUID(), nullable=False),
    sa.Column('provider', sa.String(length=50), nullable=False),
    sa.Column('model', sa.String(length=100), nullable=True),
    sa.Column('score', sa.Float(), nullable=False),
    sa.Column('max_score', sa.Float(), nullable=False),
    sa.Column('justification', sa.Text(), nullable=False),
    sa.Column('confidence', sa.Float(), nullable=False),
    sa.Column('raw_response', postgresql.JSONB(astext_type=sa.Text()), nullable=True),
    sa.Column('error', sa.Text(), nullable=True),
    sa.Column('id', sa.UUID(), nullable=False),
    sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.ForeignKeyConstraint(['answer_id'], ['answers.id'], name=op.f('fk_ai_evaluations_answer_id_answers'),
ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id', name=op.f('pk_ai_evaluations'))
    )
    op.create_index(op.f('ix_ai_evaluations_answer_id'), 'ai_evaluations', ['answer_id'], unique=False)
    # ### end Alembic commands ###


def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_index(op.f('ix_ai_evaluations_answer_id'), table_name='ai_evaluations')
    op.drop_table('ai_evaluations')
    op.drop_index(op.f('ix_results_published'), table_name='results')
    op.drop_table('results')
    op.drop_index('ix_proctor_events_timeline', table_name='proctor_events')
    op.drop_table('proctor_events')
    op.drop_index(op.f('ix_answers_session_id'), table_name='answers')
    op.drop_index(op.f('ix_answers_grade_status'), table_name='answers')
    op.drop_table('answers')
    op.drop_table('question_options')
    op.drop_index(op.f('ix_exam_sessions_status'), table_name='exam_sessions')
    op.drop_index(op.f('ix_exam_sessions_is_flagged'), table_name='exam_sessions')
    op.drop_index(op.f('ix_exam_sessions_exam_id'), table_name='exam_sessions')
    op.drop_index(op.f('ix_exam_sessions_candidate_id'), table_name='exam_sessions')
    op.drop_table('exam_sessions')
    op.drop_index(op.f('ix_exam_questions_exam_id'), table_name='exam_questions')
    op.drop_table('exam_questions')
    op.drop_index('ix_questions_pool_lookup', table_name='questions')
    op.drop_index(op.f('ix_questions_is_active'), table_name='questions')
    op.drop_table('questions')
    op.drop_index(op.f('ix_exams_status'), table_name='exams')
    op.drop_table('exams')
    op.drop_index(op.f('ix_users_role'), table_name='users')
    op.drop_index(op.f('ix_users_email'), table_name='users')
    op.drop_index(op.f('ix_users_access_status'), table_name='users')
    op.drop_table('users')
    op.drop_index(op.f('ix_subjects_code'), table_name='subjects')
    op.drop_table('subjects')
    # ### end Alembic commands ###
```

a17b4c9de204_word_bounds_thumbnails_ocr.py

backend/alembic/versions/a17b4c9de204_word_bounds_thumbnails_ocr.py · 37 lines

```python
"""answer word bounds, image thumbnails, OCR pre-pass

Revision ID: a17b4c9de204
Revises: f0c6eeb2272c
Create Date: 2026-09-01 10:12:03.881204
"""
from __future__ import annotations

from collections.abc import Sequence

import sqlalchemy as sa
from alembic import op

revision: str = 'a17b4c9de204'
down_revision: str | None = 'f0c6eeb2272c'
branch_labels: str | Sequence[str] | None = None
depends_on: str | Sequence[str] | None = None


def upgrade() -> None:
    op.add_column('questions', sa.Column('min_words', sa.Integer(), nullable=True))
    op.add_column('questions', sa.Column('max_words', sa.Integer(), nullable=True))

    op.add_column('answers', sa.Column('image_thumb_key', sa.String(length=512), nullable=True))
    op.add_column('answers', sa.Column('ocr_text', sa.Text(), nullable=True))
    op.add_column('answers', sa.Column('ocr_confidence', sa.Float(), nullable=True))
    op.add_column('answers', sa.Column('word_count', sa.Integer(), nullable=True))


def downgrade() -> None:
    op.drop_column('answers', 'word_count')
    op.drop_column('answers', 'ocr_confidence')
    op.drop_column('answers', 'ocr_text')
    op.drop_column('answers', 'image_thumb_key')

    op.drop_column('questions', 'max_words')
    op.drop_column('questions', 'min_words')
```

f0c6eeb2272c_login_access_requests_exam_enrollments_.py

backend/alembic/versions/f0c6eeb2272c_login_access_requests_exam_enrollments_.py · 116 lines

```python
"""login access requests, exam enrollments, split names

Revision ID: f0c6eeb2272c
Revises: 03ad33e6bd09
Create Date: 2026-08-27 16:44:14.186364
"""
from __future__ import annotations

from collections.abc import Sequence

import sqlalchemy as sa
from alembic import op

revision: str = 'f0c6eeb2272c'
down_revision: str | None = '03ad33e6bd09'
branch_labels: str | Sequence[str] | None = None
depends_on: str | Sequence[str] | None = None


def upgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.create_table('login_access_requests',
    sa.Column('candidate_id', sa.UUID(), nullable=False),
    sa.Column('status', sa.Enum('pending', 'approved', 'rejected', name='login_access_status'), nullable=False),
    sa.Column('requested_at', sa.DateTime(timezone=True), nullable=False),
    sa.Column('reviewed_at', sa.DateTime(timezone=True), nullable=True),
    sa.Column('reviewed_by_id', sa.UUID(), nullable=True),
    sa.Column('review_note', sa.Text(), nullable=True),
    sa.Column('id', sa.UUID(), nullable=False),
    sa.Column('created_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.Column('updated_at', sa.DateTime(timezone=True), server_default=sa.text('now()'), nullable=False),
    sa.ForeignKeyConstraint(['candidate_id'], ['users.id'], name=op.f('fk_login_access_requests_candidate_id_users'), ondelete='CASCADE'),
    sa.ForeignKeyConstraint(['reviewed_by_id'], ['users.id'], name=op.f('fk_login_access_requests_reviewed_by_id_users'), ondelete='SET NULL'),
    sa.PrimaryKeyConstraint('id', name=op.f('pk_login_access_requests'))
    )
    op.create_index(op.f('ix_login_access_requests_candidate_id'), 'login_access_requests', ['candidate_id'], unique=True)
    op.create_index(op.f('ix_login_access_requests_status'), 'login_access_requests', ['status'], unique=False)
    op.create_table('exam_enrollments',
    sa.Column('exam_id', sa.UUID(), nullable=False),
    sa.Column('candidate_id', sa.UUID(), nullable=False),
    sa.Column('assigned_by_id', sa.UUID(), nullable=True),
    sa.Column('assigned_at', sa.DateTime(timezone=True), nullable=False),
    sa.Column('id', sa.UUID(), nullable=False),
    sa.ForeignKeyConstraint(['assigned_by_id'], ['users.id'], name=op.f('fk_exam_enrollments_assigned_by_id_users'), ondelete='SET NULL'),
    sa.ForeignKeyConstraint(['candidate_id'], ['users.id'], name=op.f('fk_exam_enrollments_candidate_id_users'), ondelete='CASCADE'),
    sa.ForeignKeyConstraint(['exam_id'], ['exams.id'], name=op.f('fk_exam_enrollments_exam_id_exams'), ondelete='CASCADE'),
    sa.PrimaryKeyConstraint('id', name=op.f('pk_exam_enrollments')),
    sa.UniqueConstraint('exam_id', 'candidate_id', name='uq_exam_enrollment')
    )
    op.create_index(op.f('ix_exam_enrollments_candidate_id'), 'exam_enrollments', ['candidate_id'], unique=False)
    op.create_index(op.f('ix_exam_enrollments_exam_id'), 'exam_enrollments', ['exam_id'], unique=False)
    # Added with a server default so the columns can be NOT NULL on a populated table;
    # the default is dropped again once the backfill below has run.
    op.add_column(
        'users',
        sa.Column('first_name', sa.String(length=80), nullable=False, server_default=''),
    )
    op.add_column(
        'users',
        sa.Column('last_name', sa.String(length=80), nullable=False, server_default=''),
    )
    # ### end Alembic commands ###

    # --- data migration ------------------------------------------------
    # Split existing full names: everything before the first space is the first name,
    # the remainder is the last name. Nothing is deleted and full_name is left intact.
```

```
    op.execute(
        """
        UPDATE users
           SET first_name = split_part(trim(full_name), ' ', 1),
               last_name  = trim(substr(trim(full_name),
                                        length(split_part(trim(full_name), ' ', 1)) + 1))
         WHERE first_name = ''
        """
    )
    op.alter_column('users', 'first_name', server_default=None)
    op.alter_column('users', 'last_name', server_default=None)

    # Existing candidates were already using the platform, so grandfather them in as
    # approved rather than locking everyone out behind a new pending gate.
    op.execute(
        """
        INSERT INTO login_access_requests
              (id, candidate_id, status, requested_at, reviewed_at, review_note,
               created_at, updated_at)
        SELECT gen_random_uuid(), u.id, 'approved', u.created_at, now(),
               'Granted automatically when login approval was introduced', now(), now()
          FROM users u
         WHERE u.role = 'candidate'
        """
    )

    # Likewise, candidates could previously see every published exam. Enrol them in the
    # exams that already exist so nobody loses access to a paper they could already sit.
    op.execute(
        """
        INSERT INTO exam_enrollments (id, exam_id, candidate_id, assigned_at)
        SELECT gen_random_uuid(), e.id, u.id, now()
          FROM exams e
          CROSS JOIN users u
         WHERE u.role = 'candidate'
        """
    )


def downgrade() -> None:
    # ### commands auto generated by Alembic - please adjust! ###
    op.drop_column('users', 'last_name')
    op.drop_column('users', 'first_name')
    op.drop_index(op.f('ix_exam_enrollments_exam_id'), table_name='exam_enrollments')
    op.drop_index(op.f('ix_exam_enrollments_candidate_id'), table_name='exam_enrollments')
    op.drop_table('exam_enrollments')
    op.drop_index(op.f('ix_login_access_requests_status'), table_name='login_access_requests')
    op.drop_index(op.f('ix_login_access_requests_candidate_id'), table_name='login_access_requests')
    op.drop_table('login_access_requests')
    # ### end Alembic commands ###
```